Lab 6: Directives, Macros & Checksum

Microprocessors (010153521) | Semester 1/2026

Duration: 3 hours (in-lab) | **Topic:** Assembler directives, AVR addressing modes, user-defined macros, and EEPROM data memory, consolidated into an Intel-HEX-style checksum integrity check – the last assembly lab before the midterm

Objectives

- Use assembler **directives** to organize a program across Flash code (`.CSEG`), a Flash data table (`.DB`), and EEPROM data (`.ESEG`) – not just `.INCLUDE` and `.ORG`, which you've already used since Lab 1.
- Contrast AVR's **addressing modes** side by side: immediate, register-direct, and indirect-with-post-increment (the `Z` register with `LPM`) to walk a table in Flash.
- Write and use your own **macro** (`.MACRO`/`.ENDM`) to eliminate repeated code – and learn the one gotcha every AVR macro writer hits eventually.
- Read persistent **EEPROM** memory at runtime and compare it against a freshly computed **checksum** – the same two's-complement scheme every Intel HEX file you've uploaded since Lab 3 already relies on.

Quick Reference: Assembler Directives You'll Use

Directive	Purpose	Example
`.EQU`	Name a constant – uses no memory, just text substitution at assemble time	`.EQU TABLE_LEN = 8`
`.CSEG`	Begin/continue the Flash *code* segment (also holds `.DB` tables read via `LPM`)	`.CSEG`
`.DB`	Define byte(s) at the current address in the current segment	`TABLE: .DB 8,4,7,5,6,1,2,3`
`.ESEG`	Begin/continue the **EEPROM** segment – a separate address space from Flash and SRAM	`.ESEG`
`.ORG`	Set the address of whatever follows, within the current segment	`.ORG 0x0000`
`.MACRO` / `.ENDM`	Define a reusable named block of instructions, expanded inline at every invocation	see the macro box below

Every directive here is scoped to *assembly time* – none of them exist once your program is running. The three segments (`.CSEG`/Flash, `.ESEG`/EEPROM, and SRAM's `.DSEG`, not used in this lab) are physically separate memories with separate address spaces: address `0x0000` in Flash and address `0x0000` in EEPROM are two completely different storage cells.

Quick Reference: Addressing Modes Side by Side

Mode	Example	What it means
Immediate	`LDI R16,0x24`	The operand *is* the value `0x24`, encoded directly in the instruction word
Register (direct)	`ADD R17,R16`	The operand is whatever a register currently holds
Direct I/O	`OUT PORTB,R16`	The operand is a fixed I/O register address
Indirect, post-increment	`LPM R16,Z+`	The address to read is held *in* the `Z` register (a pointer, not a value); read Flash at that address into `R16`, then increment `Z`

Every AVR instruction you've written through Lab 5 used one of the first three modes. Indirect addressing is the new one here: instead of the instruction naming a fixed address, a register *holds* the address – which is exactly what lets a fixed-size loop walk a table of any length without one hardcoded instruction per byte.

Quick Reference: Defining and Using a Macro

```
.MACRO CHECKSUM_STEP
    LPM   R16, Z+      ;read next table byte, advance Z
    ADD   R17, R16     ;add it into the running sum
.ENDM

;invoke it just like any instruction, as many times as you like:
CHECKSUM_STEP
CHECKSUM_STEP
```

A macro's body is *inserted fresh* into your code at every invocation – text substitution at assemble time, not a runtime call like `CALL`/`RET` (no stack use, no return address, but also no code-size savings: 10 invocations means 10 copies of the body in Flash). `CHECKSUM_STEP` above is safe to invoke any number of times because it contains no label of its own. A macro that *does* contain an internal label will fail to assemble the second time it's invoked in the same file – that label would then be defined twice (see Find the Bug 1).

Quick Reference: EEPROM Memory

EEPROM is a third, separate memory: unlike SRAM it **survives power loss**, but writes are far slower (≈3.3 ms/byte) and rated for only about **100,000 write cycles per byte** on the ATmega328P – reserve it for values that must persist, not for anything rewritten every loop pass. Reserve space with `.ESEG` exactly like a Flash table, but reading it back at runtime needs the EEPROM control registers, not `LPM`:

```
.MACRO EEPROM_READ          ;always reads EEPROM address 0x0000
WAIT_EE:
    SBIC  EECR, EEPE        ;wait for any earlier write to finish
    RJMP  WAIT_EE
    LDI   R16, 0
    OUT   EEARH, R16
    OUT   EEARL, R16        ;address = 0x0000
    SBI   EECR, EERE        ;trigger the read
```

```
    IN    @0, EEDR          ;@0 = the macro's parameter (destination register)
.ENDM
```

(`EEPE`/`EERE` are `m328Pdef.inc`'s names for what older textbooks and chips call `EEWE`/`EERE` – same register bits, newer names, exactly the kind of naming difference Lab 3 already warned you about.) Studio assembles a separate `.eep` file alongside your `.hex`; upload it with `avrdude`'s EEPROM memory type – the same `-U` syntax as Lab 3's flash write, just a different memory name:

```
avrdude -c arduino -p atmega328p -P <port> -b 115200 -U eeprom:w:yourfile.eep:i
```

Quick Reference: The Two's-Complement Checksum

$$\text{sum} = (\text{byte}_1 + \text{byte}_2 + \cdots + \text{byte}_N) \bmod 256, \qquad \text{checksum} = (256 - \text{sum}) \bmod 256$$

`checksum` is simply the two's complement of `sum` (one `NEG` instruction). This definition guarantees sum + checksum $\equiv 0 \pmod{256}$ always – so *verifying* is just "add every byte, including the checksum byte itself; the total's low byte must come out exactly `0`." This is the *exact* scheme Intel HEX records use: `avrdude` has been checking a checksum exactly like this on every record of every `.hex` upload you've done since Lab 3.

Quick Reference: Your Individual Data Table

Take the **last 8 digits** of your student ID as 8 individual byte values (`0-9` each) – this is your `TABLE`. **Worked example** (a made-up ID, do not reuse): ID `1029384756123` → last 8 digits `84756123` → table bytes `8,4,7,5,6,1,2,3`.

$$\text{sum} = 8{+}4{+}7{+}5{+}6{+}1{+}2{+}3 = 36\ (\texttt{0x24}), \qquad \text{checksum} = 256 - 36 = 220\ (\texttt{0xDC})$$

Check: $36 + 220 = 256$, low byte $= 0$. ✓ Compute *your own* table and checksum now, by hand, in Exercise 1 – before touching the keyboard.

Bill of Materials

No new parts this lab – just the **Arduino Uno R3** from previous labs. This lab reuses the Uno's **on-board LED** (`PB5`/D13) for pass/fail indication; no breadboard circuit is required.

Quick Experiments (Try It)

Try It 1: Your first macro

Define `INIT_STACK` as a macro (the four-line stack-pointer initialization you've hand-typed at the top of every program since Lab 3):

```
.MACRO INIT_STACK
    LDI   R16, HIGH(RAMEND)
    OUT   SPH, R16
    LDI   R16, LOW(RAMEND)
    OUT   SPL, R16
.ENDM
```

Invoke it once at the top of `MAIN`, build, and open the `.lst` file – confirm the single line

`INIT_STACK` expanded into the same four instructions you'd have typed by hand.

Try It 2: One indirect read, by itself

Load `Z` with your `TABLE`'s address (`LDI ZL,LOW(TABLE*2)` / `LDI ZH,HIGH(TABLE*2)` – Flash byte addresses are double the word address the label gives you, hence the `*2`), execute a single `LPM R16,Z+` by itself, and confirm `R16` holds your table's *first* byte (write it to `PORTB` and check in the simulator, or on a logic probe) before writing any loop.

In-Lab Exercises (target: 3 hours)

In-Lab Exercise 1: Derive your table and checksum by hand *(25 min)*

Using the "Your Individual Data Table" box above, write out your 8 table bytes and compute `sum` and `checksum` by hand in your notebook, showing every step. Double-check by adding `sum + checksum` yourself and confirming the low byte is exactly `0` – this is the same value your program will compute in Exercise 3, so an arithmetic slip here will cost you later.

In-Lab Exercise 2: Program structure with directives *(35 min)*

Set up your Microchip Studio project with:

- A `.CSEG` table (`TABLE: .DB` your 8 bytes from Exercise 1) and `.EQU TABLE_LEN = 8`.
- An `.ESEG` reservation (`REF_CHECKSUM: .DB` your hand-computed checksum from Exercise 1).
- Your `INIT_STACK` macro from Try It 1, invoked once in `MAIN`.

Build the project – Studio produces *both* a `.hex` and a `.eep` file. Upload **both** with `avrdude` (flash exactly as in Lab 3, EEPROM using the Quick Reference box's `eeprom` memory type). Do *not* write any checksum-verification logic yet – this exercise is only about getting the segments, directives, and both uploads right.

In-Lab Exercise 3: Runtime checksum and verification *(85 min)*

Write your own `CHECKSUM_STEP` macro (following the Quick Reference pattern) and use it inside a loop of `TABLE_LEN` iterations to compute the actual checksum over your Flash `TABLE` at runtime, using `Z` + `LPM` indirect addressing. Use the *given* `EEPROM_READ` macro (Quick Reference box above) to read `REF_CHECKSUM` back from EEPROM. Compare the two values (`CP`); if they match, light the on-board LED (`PB5`) steady; if they don't, leave it off.

- **Trace/predict first, then verify:** before running it, predict what your program's internally computed `sum + checksum` should equal, and what the LED should show. Then build, upload, and confirm.

In-Lab Exercise 4: Break it on purpose *(30 min)*

Deliberately change *one* byte in your Flash `TABLE`, re-assemble, and re-upload **flash only** (leave the EEPROM reference untouched). Confirm the LED now correctly reports failure. Change the byte back, re-upload, and confirm the LED reports success again.

- In your notebook, explain in one or two sentences why this exact mechanism – a stored reference checksum compared against a freshly computed one – is how real bootloaders and firmware updates detect a corrupted transfer.

AI Collaboration

Try these prompts with an AI tool:

- "What's the actual difference between an AVR macro (`.MACRO`/`.ENDM`) and a subroutine (`CALL`/`RET`) – when would I choose one over the other?"
- "Why does a fixed internal label inside an AVR macro cause a 'duplicate label' error the second time the macro is invoked, and how do real AVR programs work around it?"
- "What is EEPROM's typical write endurance on an ATmega328P, and why does that affect what you should and shouldn't store there?"

Critical check – verify, don't just accept: If your Exercise 3 LED shows failure on the first try, verify your hand-computed checksum (Exercise 1) against the runtime value byte by byte before assuming your code is wrong – a surprising fraction of first failures turn out to be an arithmetic slip in the *hand* calculation, not a bug in the program.

Know the limit: AI tools frequently use "checksum" and "CRC" interchangeably – they are *not* the same algorithm and do not catch the same errors. A simple sum-based checksum like this lab's misses several common multi-bit corruption patterns that a CRC would catch; don't let an AI's phrasing imply your checksum has CRC-level protection when asking it for help.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Compute a checksum by hand

For the 6-byte table `0x1A, 0x03, 0xFF, 0x40, 0x02, 0x1C`, compute `sum` and the two's-complement `checksum` by hand, showing your work. Verify by confirming `sum + checksum` has a low byte of `0`.

Find the Bug 1: A macro invoked twice

A student wrote the `EEPROM_READ` macro exactly as given in the Quick Reference box (with its internal `WAIT_EE` label) and invoked it *twice* in the same program – once to read a reference checksum, and once to read a second stored calibration byte at a different address. Predict the exact category of error the assembler reports, and explain – in terms of what a macro invocation actually does to your source code – why it happens on the *second* invocation specifically, not the first.

Write Code on Paper 1: A parameterized macro

On paper, without a computer, simulator, or AI tool, write a macro `TOGGLE_LED` that takes one parameter (a bit number, `0-7`) and toggles that bit of `PORTB` – *without* first reading the pin to figure out which state it's currently in. (Hint: re-read the ATmega328P port section on what writing a `1` to a `PINx` bit does.)

Draw on Paper 1: Draw your program's memory map

Draw a memory map of your assembled Exercise 3 program: separate boxes for Flash (showing `MAIN`'s code and `TABLE`, with its address) and EEPROM (showing `REF_CHECKSUM`, with its address) as two *physically separate* address spaces. For each box, label which instruction and

which addressing mode your program uses to reach it (e.g. "`LPM Z+`, indirect, reads `TABLE`").

Reflection

This is the last assembly lab before the midterm – after this, the course moves to C. In one short paragraph, look ahead: which of today's five concepts (directives, addressing modes, macros, EEPROM, checksums) do you expect to become *invisible* in C, handled automatically by the compiler, and which will you still control directly yourself, just through different syntax? Consider specifically: assembler segments (`.CSEG`/`.ESEG`) versus a compiler's memory sections, indirect addressing versus C pointers, `.MACRO` versus `#define`, and EEPROM access via raw registers versus a library call. Is there anything on today's list that C *doesn't* make easier?